НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ

«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Факультет інформатики та обчислювальної техніки

Звіт до практичної роботи №3

З дисципліни «Економіка ІТ-індустрії та підприємництво»

Виконали:

Студенти 4 курсу, гр. ІП-24

Малий Олександр,

Молебна Марія,

Досмухамбетов Назар

Київ – 2025

Тема: Аналіз функціональних точок

Мета: Навчитися оцінювати економічні характеристики програмних продуктів на основі аналізу функціональних точок.

ЗАВДАННЯ

1. Проаналізувати власний програмний застосунок та скласти перелік та навести докладний опис всіх внутрішніх логічних (ILF) і зовнішніх інтерфейсних (EIF) файлів, а також всіх транзакцій (EI, EO, EQ).
2. Виконати оцінки кількості RET і DET для внутрішніх логічних (ILF) і зовнішніх інтерфейсних (EIF) файлів, а також оцінки кількості FTR і DET для зовнішніх вводів (EI), виведень (EO) і запитів (EQ), оцінити складність та кількість ненормованих функціональних точок.
3. Провести аналізу ступенів впливу основних характеристик системи.
4. Розрахувати нормовану кількість функціональних точок та економічних характеристик.

ВИКОНАННЯ

Для виконання цієї роботи було використано курсову роботу: https://github.com/JessFreak/ServiceStation. Переважно описано функції бекенда, оскільки їх легше документувати.

1. Аналіз програмного застосунку ServiceStation: перелік та опис функціональних елементів

На основі аналізу розроблених контролерів серверної частини застосунку було ідентифіковано наступні елементи функціональних точок.

1.1. Внутрішні логічні файли (ILF — Internal Logical Files)

Це групи даних, які зберігаються в базі даних системи (через Prisma ORM) та підтримуються транзакціями застосунку.

1. **ILF «User» (Користувачі)**
    - Опис: Зберігає облікові дані користувачів.
    - Деталі (DET): Ідентифікатор (id), email, хеш пароля, роль (ROLE: ADMIN, WORKER, USER), refresh token.
    - Обґрунтування: Використовується в AuthController та UsersController.
2. **ILF «Vehicle» (Транспортні засоби)**
    - Опис: Інформація про автомобілі клієнтів.
    - Деталі (DET): Ідентифікатор, марка, модель, VIN-код, номерний знак, посилання на власника (userId).
    - Обґрунтування: Керується через UsersController (методи addVehicle, updateVehicle).
3. **ILF «Service» (Послуги)**
    - Опис: Каталог послуг, що надаються СТО.
    - Деталі (DET): Назва послуги, опис, ціна, тривалість.
    - Обґрунтування: Керується через ServiceController.
4. **ILF «Order» (Замовлення)**
    - Опис: Основна сутність процесу обслуговування.
    - Деталі (DET): Дата запису, статус (Pending, In Progress, Done, Cancelled), посилання на клієнта, авто, послугу та призначеного майстра (workerId).
    - Обґрунтування: Керується через OrderController.

1.2. Зовнішні інтерфейсні файли (EIF — External Interface Files)

Дані, які використовуються системою, але керуються ззовні.

1. **EIF «Google OAuth Identity»**

- Опис: Профіль користувача в системі Google.
- Обґрунтування: Контролер AuthController має методи google та googleAuthCallback (використовує GoogleOauthGuard). Система отримує підтвердження особи від Google, але не змінює пароль чи дані акаунта Google.

1.3. Зовнішні входи (EI — External Inputs)

Транзакції, які надходять ззовні та оновлюють внутрішні дані (ILF). Складено на основі методів POST, PATCH, DELETE у контролерах.

- **Група: Автентифікація та Користувачі (AuthController, UsersController)**
  1. **Реєстрація (Register):** Створення нового користувача (POST /auth/register).
  2. **Вхід у систему (Login):** Отримання доступу та генерація токена (POST /auth/login).
  3. **Зміна пароля:** Оновлення пароля поточного користувача (PATCH /auth/password).
  4. **Видалення акаунта:** Видалення власного запису (DELETE /auth/me).
  5. **Створення користувача (Адмін):** Адміністратор створює користувача вручну (POST /users).
  6. **Зміна ролі (Адмін):** Призначення ролі (наприклад, WORKER) іншому користувачу (PATCH /users/:userId).
  7. **Оновлення профілю:** Користувач змінює свої дані (PATCH /users).
- **Група: Транспортні засоби (UsersController)**

1. 8 **Додавання авто:** Клієнт додає новий автомобіль у свій гараж (POST /users/vehicles).
2. **Редагування авто:** Оновлення даних про автомобіль (PATCH /users/vehicles/:vehicleId).
3. **Видалення авто:** Вилучення автомобіля з профілю (DELETE /users/vehicles/:vehicleId).

- **Група: Послуги (ServiceController)**
  1. **Створення послуги:** Адміністратор додає нову послугу в прайс (POST /services).
  2. **Оновлення послуги:** Зміна ціни або назви послуги (PATCH /services/:serviceId).
  3. **Видалення послуги:** Видалення позиції з каталогу (DELETE /services/:serviceId).
- **Група: Замовлення (OrderController, UsersController)**
  1. **Створення замовлення:** Клієнт записується на сервіс (POST /orders).
  2. **Скасування замовлення:** Клієнт скасовує власне замовлення (POST /users/orders/:orderId/cancel).
  3. **Призначення майстра:** Адміністратор призначає робітника на замовлення (PATCH /orders/:orderId/worker).
  4. **Зміна статусу замовлення:** Робітник оновлює стан виконання робіт (PATCH /orders/:orderId/status).

1.4. Зовнішні запити (EQ — External Inquiries)

Транзакції, що передбачають вибірку даних без зміни їх стану. Складено на основі методів GET.

1. **Перегляд власного профілю:** Отримання даних про поточного користувача (GET /auth/me).

2. **Перевірка наявності пароля:** Технічний запит для логіки OAuth (GET /auth/hasPassword).
3. **Перегляд каталогу послуг:** Отримання списку всіх послуг із фільтрацією (GET /services + ServiceQueryDTO).
4. **Перегляд списку користувачів:** Адміністратор переглядає всіх зареєстрованих (GET /users).
5. **Перегляд власних авто:** Клієнт отримує список своїх транспортних засобів (GET /users/vehicles).
6. **Перегляд історії замовлень (Клієнт):** Клієнт переглядає свої минулі записи (GET /users/orders).
7. **Перегляд всіх замовлень (Адмін):** Адміністратор переглядає журнал записів з фільтрами по юзеру чи послузі (GET /orders).
8. **Перегляд призначених замовлень (Робітник):** Майстер переглядає замовлення, призначені саме йому (GET /orders/assigned).

1.5. Зовнішні виходи (EO — External Outputs)

Транзакції, що генерують дані з обчисленнями або формують специфічні звіти.

1. **Логаут (Logout):** Процес завершення сесії, який може включати інвалідацію токена та повернення відповідного статусу (POST /auth/logout).
2. **Формування відповіді про замовлення (Order Response):** Методи отримання замовлень використовують OrderMapper, який трансформує дані з БД у зручний для клієнта формат (обчислюючи або форматуючи поля), що можна класифікувати як вихід даних.

2. Оцінка складності та розрахунок ненормованих функціональних точок (UFP)

На цьому етапі для кожного виділеного елемента визначається його складність (Низька, Середня, Висока) на основі кількості елементів даних (DET), типів записів (RET) та файлів, до яких здійснюється звернення (FTR).

2.1. Оцінка складності функцій даних (ILF та EIF)

Для внутрішніх логічних файлів (ILF) та зовнішніх інтерфейсних файлів (EIF) складність визначається за кількістю DET (полів) та RET (підгруп даних). Оскільки використовується реляційна база даних (PostgreSQL) і кожна сутність є окремою таблицею, приймаємо RET = 1 для всіх ILF.

Тип	**Функціональний елемент**	**DET (прибл.)**	**RET**	**Складність**	**Бали (UFP)**
ILF	User (Користувачі)	6	1	Low	7
ILF	Vehicle (Автомобілі)	6	1	Low	7
ILF	Service (Послуги)	5	1	Low	7
ILF	Order (Замовлення)	8	1	Low	7
EIF	Google OAuth Identity	3	1	Low	5

	Всього за дані:				**33**

2.2. Оцінка складності транзакційних функцій (EI, EQ, EO)

Для транзакцій складність залежить від кількості FTR (кількість таблиць/файлів, які зачіпаються) та DET (кількість полів, що передаються).

Зовнішні входи (EI — External Inputs)

Більшість CRUD-операцій (створення, редагування, видалення) зачіпають 1 файл (FTR=1) і мають невелику кількість полів (<15), тому мають низьку складність. Операція "Створення замовлення" є складнішою, оскільки посилається на Users, Vehicles та Services (FTR > 2).

ID	**Транзакція (EI)**	**FTR**	**DET**	**Складність**	**Бали**
EI1	Реєстрація	1	4	Low	3
EI2	Вхід у систему (Login)	1	2	Low	3
EI3	Зміна пароля	1	2	Low	3
EI4	Видалення акаунта	1	1	Low	3
EI5	Створення користувача (Адмін)	1	5	Low	3

EI6	Зміна ролі	1	2	Low	3
EI7	Оновлення профілю	1	3	Low	3
EI8	Додавання авто	1	5	Low	3
EI9	Редагування авто	1	5	Low	3
EI10	Видалення авто	1	1	Low	3
EI11	Створення послуги	1	4	Low	3
EI12	Оновлення послуги	1	4	Low	3
EI13	Видалення послуги	1	1	Low	3
EI14	Створення замовлення	3	5	Average	4
EI15	Скасування замовлення	1	1	Low	3

EI16	Призначення майстра	1	2	Low	3
EI17	Зміна статусу замовлення	1	2	Low	3
	Всього EI:				**52**

Транзакція оновлює ILF Order, але перевіряє/посилається на ILF User, Vehicle та Service. FTR=3, DET=5-15. Згідно з табл. 5, це відповідає рівню Average.

Зовнішні запити (EQ — External Inquiries)

Запити на читання даних. Складні запити (фільтрація замовлень) можуть зачіпати кілька таблиць (JOINs).

ID	**Транзакція (EQ)**	**FTR**	**DET**	**Складність**	**Бали**
EQ1	Перегляд профілю	1	6	Low	3
EQ2	Перевірка пароля	1	1	Low	3
EQ3	Каталог послуг	1	5	Low	3
EQ4	Список користувачів	1	5	Low	3

EQ5	Список авто	1	6	Low	3
EQ6	Історія замовлень (Клієнт)	3	8	Average	4
EQ7	Всі замовлення (Адмін)	3	8	Average	4
EQ8	Призначені замовлення	3	8	Average	4
	Всього EQ:				**27**

Виведення замовлення вимагає даних з таблиць User, Vehicle, Service (Join). FTR=3, DET=6-19. Згідно з табл. 5 (для EQ вхідна частина оцінюється як EI), це відповідає рівню Average.

Зовнішні виходи (EO — External Outputs)

ID	**Транзакція (EO)**	**FTR**	**DET**	**Складність**	**Бали**
EO1	Логаут (Logout)	1	1	Low	4
EO2	Order Response (Форматування)	1	8	Low	4

	Всього EO:				**8**

2.3. Розрахунок загальної кількості ненормованих функціональних точок (UFPC)

Використовуючи формулу сумування всіх балів:

UFPC = ILF + EIF + EI + EQ + EO

Підставимо отримані значення:

1. ILF: 7 * 4 = 28
2. EIF: 5*1 = 5
3. EI: 3*16 + 4 * 1 = 52
4. EQ: 3 * 5 + 4 * 3 = 27
5. EO: 4 * 2 = 8

UFPC = 28 + 5 + 52 + 27 + 8 = 120

Результат: Ненормований розмір програмного продукту становить **120** функціональних точок.

3. Аналіз ступенів впливу характеристик та розрахунок VAF

Для визначення нормуючого фактора було оцінено 14 загальних системних характеристик (GSC) за шкалою від 0 (немає впливу) до 5 (сильний вплив).

Таблиця 3.1 — Оцінка характеристик системи

№	Характеристика	Оцінка	Коментар
1	Обмін даними	3	Використання REST API та HTTP.

2	Розподілена обробка	3	Архітектура Клієнт-Сервер.
3	Продуктивність	0	Спеціальні вимоги відсутні.
4	Обмеження по апаратурі	0	Стандартне середовище.
5	Транзакційне навантаження	1	Помірне, без піків.
6	Взаємодія з користувачем	5	Повністю інтерактивний інтерфейс.
7	Ергономіка	3	Зручний Web-інтерфейс (React).
8	Інтенсивність зміни даних	3	Часті оновлення даних в БД.
9	Складність обробки	1	Стандартна логіка та валідація.

10	Повторне використання	1	Часткова модульність коду.
11	Зручність інсталяції	2	Стандартне розгортання (npm).
12	Зручність адміністрування	0	Інструменти відсутні.
13	Можливість портування	2	Кросплатформність (Node.js).
14	Гнучкість	3	Легка модифікація компонентів.
	Всього (TDI)	**27**	

Розрахунок нормуючого фактора (VAF)

Сумарний ступінь впливу (TDI) дорівнює 27.

Нормуючий фактор розраховується за формулою:

$$VAF = (TDI \cdot 0.01) + 0.65$$

$$VAF = (27 \cdot 0.01) + 0.65 = 0.92$$

Результат: Нормуючий фактор становить **0.92**.

4. Розрахунок нормованої кількості функціональних точок та економічних характеристик

На цьому етапі визначається кінцевий розмір програмного продукту з урахуванням характеристик системи, а також прогнозується обсяг програмного коду.

4.1. Розрахунок нормованої кількості функціональних точок (AFPC)

Нормована (скоригована) кількість функціональних точок розраховується за формулою:

$$AFPC = UFPC \cdot VAF$$

Де:

- **UFPC** (Unadjusted Function Point Count) — ненормована кількість функціональних точок, розрахована в п. 2. Результат: **120**.
- **VAF** (Value Adjustment Factor) — нормуючий фактор, розрахований в п. 3. Результат: **0.92**.

Підставимо значення у формулу:

$$AFPC = 120 \cdot 0.92 = 110.4$$

Результат: Функціональний розмір застосунку ServiceStation складає **110.4** нормованих функціональних точок.

4.2. Оцінка кількості рядків вихідного коду (SLOC)

Однією з основних економічних характеристик є оцінка розміру коду. Вона виконується шляхом множення нормованої кількості функціональних точок на множник мови програмування (LM — Language Multiplier).

$$SLOC = AFPC \cdot LM$$

Оскільки проєкт реалізовано з використанням сучасних високорівневих мов та фреймворків (**TypeScript**, **NestJS**, **React**), обираємо відповідний коефіцієнт. Для мов групи JavaScript/TypeScript середнє значення LM становить приблизно **53** рядки коду на одну функціональну точку (за статистикою QSM/Capers Jones).

QSM SLOC/FP Data				
Language	**Avg**	**Median**	**Low**	**High**
ABAP (SAP) *	28	18	16	60
ASP*	51	54	15	69
Assembler *	119	98	25	320
Brio +	14	14	13	16
C *	97	99	39	333
C++ *	50	53	25	80
C# *	54	59	29	70
COBOL *	61	55	23	297
Cognos Impromptu Scripts +	47	42	30	100
Cross System Products (CSP) +	20	18	10	38
Cool:Gen/IEF *	32	24	10	82
Datastage	71	65	31	157
Excel *	209	191	131	315
Focus *	43	45	45	45
FoxPro	36	35	34	38
HTML *	34	40	14	48
J2EE *	46	49	15	67
Java *	53	53	14	134
JavaScript *	47	53	31	63
JCL *	62	48	25	221
LINC II	29	30	22	38
Lotus Notes *	23	21	19	40
Natural *	40	34	34	53
.NET *	57	60	53	60
Oracle *	37	40	17	60
PACBASE *	35	32	22	60
Perl *	24	15	15	60
PL/I *	64	80	16	80

Розрахунок:

$$SLOC = 110.4 \cdot 53 \approx 5851$$